

**LAPORAN PRAKTIKUM**  
**PEMROGRAMAN BERORIENTASI OBJEK**  
**MODUL 04 – Enkapsulasi**



CUT GEUBRINA DWI MAHARANI  
103132400034  
S1SD-05-01

**PROGRAM STUDI SAINS DATA**  
**FAKULTAS INFORMATIKA**  
**TELKOM UNIVERSITY PURWOKERTO**  
**2026**

## A. Dasar Teori

Berfokus pada tiga konsep utama OOP yaitu modifier, constructor, package. Enkapsulasi diimplementasikan dengan memanfaatkan access modifier private untuk menyembunyikan atribut internal kelas dari akses luar, yang mana manipulasi data hanya dapat dilakukan dengan method getter dan setter. Penggunaan non-access modifier seperti static dan final, serta constructor overloading yang memungkinkan pembuatan beberapa method pembentuk objek dengan variasi parameter. Selain itu, konsep package diterapkan sebagai pengelompokan kelas dalam struktur folder terorganisir untuk mempermudah manajemen kode serta menghindari konflik penamaan

## B. Guided

### 1. [Manusia & Demo Manusia]

```
public class Manusia {  
    //definisi atribut  
    private String nama;  
    private int umur;  
  
    //4.2 constructor  
    public Manusia(){};  
    public Manusia(String nama){  
        this.nama = nama;  
    }  
    public Manusia(String nama, int umur){  
        this.nama = nama;  
        this.umur = umur;  
    }  
  
    // Mahasiswa objManusia = new Manusia();  
  
    //method setter
```

```
public void setNama(String a){
    nama = a;
}

public void setUmur(int umur){
    this.umur = umur;
}

//method getter
public String getNama(){
    return nama;
}

public int getUmur(){
    return umur;
}
}

public class DemoManusia {
    public static void main(String[] args) {

        Manusia arrMns[] = new Manusia[3];

        //Constreuctor pertama
        Manusia objMns1 = new Manusia();

        // kedua
        Manusia objMns2 = new Manusia("John");
```

```

//ketiga
Manusia objMns3 = new Manusia("Baruji", 44);

arrMns[0] = objMns1;
arrMns[1] = objMns2;
arrMns[2] = objMns3;

for (int i = 0; i<3; i++) {
    System.out.println("Nama: " + arrMns[i].getNama());
    System.out.println("Umur: " + arrMns[i].getUmur());
    System.out.println();
}
}
}

```

Output :

```

Nama: null
Umur: 0

Nama: John
Umur: 0

Nama: Baruji
Umur: 44

```

## 2. [HargaPulsa, HargaToken, Main]

```

public class HargaPulsa {
    public void info(){
        System.out.println("Harga Pulsa");
        System.out.println("Rp. 5000");
        System.out.println("Rp. 10000");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
    }
}

```

```

}

public class HargaToken {
    public void info(){
        System.out.println("Harga Token");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
        System.out.println("Rp. 200000");
        System.out.println("Rp. 500000");
    }
}

public class Main {
    public static void main(String[] args){
        HargaToken objectToken = new HargaToken();
        objectToken.info();
        HargaPulsa objectPulsa = new HargaPulsa();
        objectPulsa.info();
    }
}

```

Output :

```

Harga Token
Rp. 20000
Rp. 50000
Rp. 100000
Rp. 200000
Rp. 500000
Harga Pulsa
Rp. 5000
Rp. 10000
Rp. 20000
Rp. 50000
Rp. 100000

```

C. Unguided

## Soal Unguided – Modul 4: Enkapsulasi

**Sistem Pencatatan Dataset**

Kamu sedang membangun program kecil untuk mencatat dan memeriksa kualitas dataset sebelum dianalisis, seperti tahap awal pengecekan *missing value*. Setiap dataset punya nama, jumlah baris, jumlah kolom, dan jumlah sel yang kosong (*missing*).

## Petunjuk Umum

---

Buat **3 package** dengan struktur berikut:

```
Source Packages
├── model
│   └── Dataset.java
├── laporan
│   └── LaporanDataset.java
└── main
    └── Main.java
```

### 1. Class `Dataset` (package `model`)

**Atribut** (implementasikan enkapsulasi):

| Atribut                    | Tipe                | Keterangan               |
|----------------------------|---------------------|--------------------------|
| <code>nama</code>          | <code>String</code> | Nama dataset             |
| <code>jumlahBaris</code>   | <code>int</code>    | Banyak baris (observasi) |
| <code>jumlahKolom</code>   | <code>int</code>    | Banyak kolom (fitur)     |
| <code>jumlahMissing</code> | <code>int</code>    | Banyak sel yang kosong   |

**Ketentuan tambahan:**

- Buat **konstanta** `BATAS_MISSING` bernilai `5.0` (persen). Dataset dengan missing diatas batas ini dianggap perlu dibersihkan.
- Buat **variabel milik class** yang menghitung total objek `Dataset` yang telah dibuat, beserta **method** untuk ambil nilainya tanpa perlu membuat objek.

**Constructor** (ada 3, overloading):

1. Tanpa parameter
2. Hanya parameter nama
3. Lengkap: nama, jumlahBaris, jumlahKolom, jumlahMissing

Setiap constructor harus menambah hitungan total dataset.

**Method:**

- Getter dan setter untuk semua atribut.
- Pada setter jumlahBaris, jumlahKolom, dan jumlahMissing, buat agar nilai tidak berubah.
- getPersentaseMissing() mengembalikan double: persentase sel kosong dibanding total sel ( $\text{baris} \times \text{kolom}$ ). Jika total sel = 0, kembalikan 0.
- perluDibersihkan() mengembalikan boolean: true jika persentase missing melebihi BATAS\_MISSING.

## 2. Class LaporanDataset (package laporan)

Buat method cetak yang menerima satu objek Dataset dan menampilkan laporannya dengan format:

```
=== Laporan Dataset ===
Nama          : Titanic
Jumlah Baris  : 891
Jumlah Kolom  : 12
Missing       : 866 sel (8.10%)
Status        : Perlu dibersihkan
```

Status berisi Perlu dibersihkan atau Bersih sesuai hasil dari perluDibersihkan().

## 3. Class Main (package main)

Buat tiga objek, masing-masing dengan constructor berbeda:

| Objek | Cara pembuatan | Data |
|-------|----------------|------|
|       |                |      |

|   |                                                      |                                           |
|---|------------------------------------------------------|-------------------------------------------|
| 1 | Constructor tanpa parameter, lalu isi memakai setter | Titanic, 891 baris, 12 kolom, 866 missing |
| 2 | Constructor 1 parameter                              | Wine Quality                              |
| 3 | Constructor lengkap                                  | Iris, 150 baris, 5 kolom, 0 missing       |

Simpan ketiganya dalam **array**, cetak laporannya dengan perulangan, lalu tampilkan di akhir:

Total dataset dibuat : 3

Jawaban :

```
package model;

public class Dataset {

    //Atribut private

    private String nama;

    private int jumlahBaris;

    private int jumlahKolom;

    private int jumlahMissing;


    //Class variable

    public static final double BATAS_MISSING = 5.0;

    private static int totalDataset = 0;


    //1.Co structor tanpa parameter

    public Dataset(){

        this.nama = " ";

        this.jumlahBaris = 0;

        this.jumlahKolom = 0;

        this.jumlahMissing = 0;

    }

}
```



```
        totalDataset++;
    }

    //2. Constructor dengan 1 parameter
    public Dataset(String nama){
        this.nama = "";
        this.jumlahBaris = 0;
        this.jumlahKolom = 0;
        this.jumlahMissing = 0;
        totalDataset++;
    }

    //3. Constructor lengkap
    public Dataset(String nama, int jumlahBaris, int jumlahKolom, int jumlahMissing){
        this.nama = nama;
        setJumlahBaris(jumlahBaris);
        setJumlahKolom(jumlahKolom);
        setJumlahMissing(jumlahMissing);
        totalDataset++;
    }

    //Static method untuk mendapatkan total dataset yang dibuat
    public static int getTotalDataset(){
        return totalDataset;
    }

    //Getter & Setter
    public String getNama(){
```

```
        return nama;
    }

    public void setName(String nama){
        this.nama = nama;
    }

    public int getJumlahBaris(){
        return jumlahBaris;
    }

    public void setJumlahBaris(int jumlahBaris){
        if(jumlahBaris >= 0) {
            this.jumlahBaris = jumlahBaris;
        }
    }

    public int getJumlahKolom(){
        return jumlahKolom;
    }

    public void setJumlahKolom(int jumlahKolom){
        if(jumlahKolom >= 0){
            this.jumlahKolom = jumlahKolom;
        }
    }

    public int getJumlahMissing(){
```

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> return jumlahMissing;  }  public void setJumlahMissing(int jumlahMissing){     if(jumlahMissing &gt;= 0){         this.jumlahMissing = jumlahMissing;     } }  public double getPersentaseMissing(){     int totalSel = jumlahBaris * jumlahKolom;     if(totalSel == 0){         return 0.0;     }     return ((double) jumlahMissing/totalSel) * 100.0; }  public boolean perluDibersihkan(){     return getPersentaseMissing() &gt; BATAS_MISSING; } } </pre> |
| <pre> class LaporanDataset {      public void cetak(Dataset ds){         String status = ds.perluDibersihkan() ? "Perlu dibersihkan" : "Bersih";          System.out.println("=== Laporan Dataset ===");         System.out.println("Nama      : " + ds.getNama());         System.out.println("Jumlah Baris : " + ds.getJumlahBaris());     } } </pre>                                                                                                                |

```

        System.out.println("Jumlah Kolom : " + ds.getJumlahKolom());

        System.out.printf("Missing      : %d sel (%.2f%%)%n", ds.getJumlahMissing(),
ds.getPersentaseMissing());

        System.out.println("Status      : " + status);

        System.out.println();
    }
}

```

```

public class Main {

    public static void main(String[]args){

        Dataset ds1 = new Dataset();
        ds1.setNama("Titanic");
        ds1.setJumlahBaris(891);
        ds1.setJumlahKolom(12);
        ds1.setJumlahMissing(866);


        Dataset ds2 = new Dataset ("Wine Quality");


        Dataset ds3 = new Dataset ("Iris", 150, 5, 0);


        Dataset[] daftarDataset = {ds1, ds2, ds3};


        LaporanDataset laporan = new LaporanDataset();
        for (Dataset ds : daftarDataset){
            laporan.cetak(ds);
        }

        System.out.println("Total dataset dibuat : " + Dataset.getTotalDataset());
    }
}

```

Output :

```
=== Laporan Dataset ===
Nama          : Titanic
Jumlah Baris  : 891
Jumlah Kolom  : 12
Missing       : 866 sel (8,10%)
Status        : Perlu dibersihkan

=== Laporan Dataset ===
Nama          : Wine Quality
Jumlah Baris  : 0
Jumlah Kolom  : 0
Missing       : 0 sel (0,00%)
Status        : Bersih

=== Laporan Dataset ===
Nama          : Iris
Jumlah Baris  : 150
Jumlah Kolom  : 5
Missing       : 0 sel (0,00%)
Status        : Bersih

Total dataset dibuat : 3
```

Penjelasan :

#### 1. File Dataset.java

Enkapsulasi (private) dibuat agar variabel nama, jumlahBaris, jumlahKolom, jumlahMissing tidak bisa diubah langsung secara asal dari luar class.

BATAS\_MISSING dibuat public agar bisa diakses dari mana saja dengan nilai konstan 5.0%. variabel totalDataset dibuat untuk menghitung jumlah dataset yang telah dibuat. Constructor overloading menyediakan 3 cara berbeda untuk membuat objek yaitu tanpa parameter, 1 parameter dan lengkap lalu jika objek baru dibuat maka variabel totalDataset akan bertambah 1. getTotalDataset memungkinkan untuk mengambil total dataset tanpa harus membuat instance objek. Validasi setter digunakan agar memastikan nilai baris, kolom, atau missing tidak bernilai negatif ( $\geq 0$ ). getPersentaseMissing berguna menghitung total sel dengan mengalikan nilai baris dan kolom yang hasilnya bilangan desimal dengan type casting, setelah pengecekan error pada pembagian. perluDibersihkan berguna mengembalikan boolean jika persentase missing melebihi 5.0%

2. File LaporanDataset.java

System.out.printf memformat tampilan teks agar hasil angka jumlahMissing tercetak desimal 2 digit dibelakang koma + simbol %. “?” digunakan sebagai if-else untuk menentukan isi variabel status

3. File Main.java

Menghubungkan semua bagian package model dan laporan ke dalam alur eksekusi utama dan menyimpan semua objek kedalam array Dataset[] agar pencetakan laporan dapat dilakukan secara otomatis menggunakan perulangan

D. Kesimpulan

Praktikum ini menyimpulkan bahwa konsep enkapsulasi berhasil digunakan sebagai pembatasan hak akses menggunakan modifier private, sehingga keamanan data terjamin melalui mekanisme data hiding yang terkontrol dengan getter dan setter. Selain itu teknik constructor overloading memberikan fleksibilitas tinggi dalam proses pembuatan objek. Package juga terbukti mempermudah pengelompokan kode program menjadi jelas dan sistematis sehingga meminimalisir potensi bentrok nama kelas